

Ex. No: 04

Image Translation Using OpenCV

Name : ASWINI D ****Reg. No : ****212225240015 **Slot No :** OD1199

Image Translation

Write a Python program using OpenCV to perform image translation on the "**Given image**".

The program should:

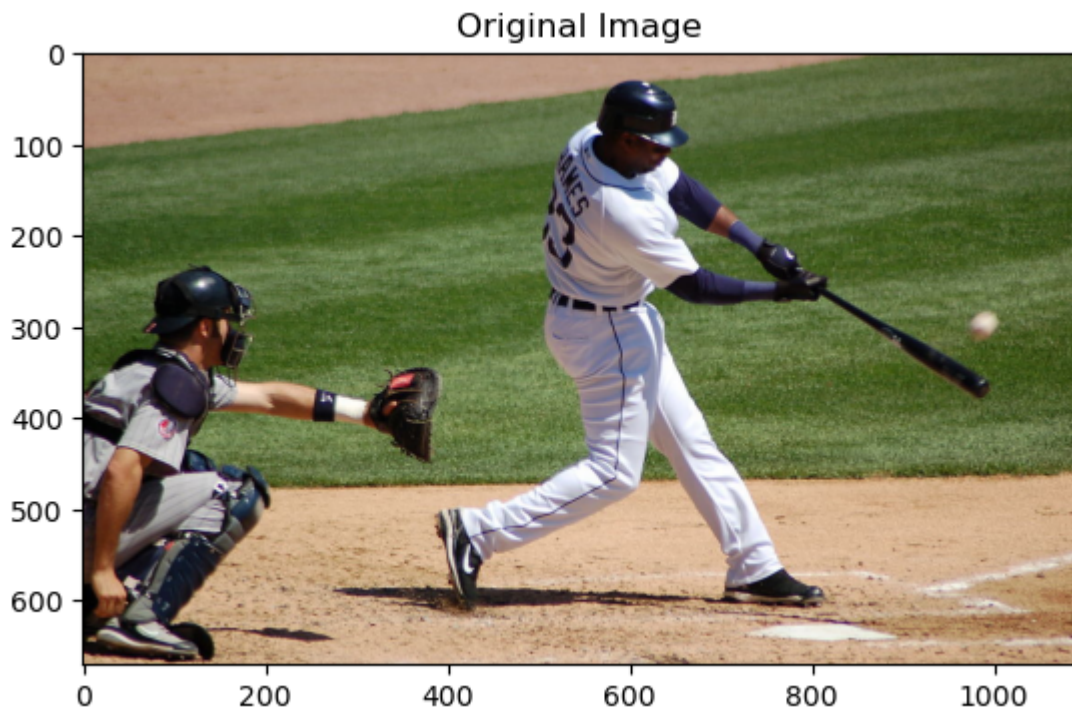
1. Import `cv2` , `numpy` , and `matplotlib.pyplot` .
2. Read the image "**given**" in color mode.
3. Create a **translation matrix** that moves the image **50 pixels to the right** and **80 pixels down**.
4. Apply the translation
5. Display the following using a **1 × 2 layout** in Matplotlib:
 - **Original Image**
 - **Translated Image**

```
In [1]: # Import
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: # Read the image using OpenCV
image = cv2.imread('baseball.jpg')
image.shape
```

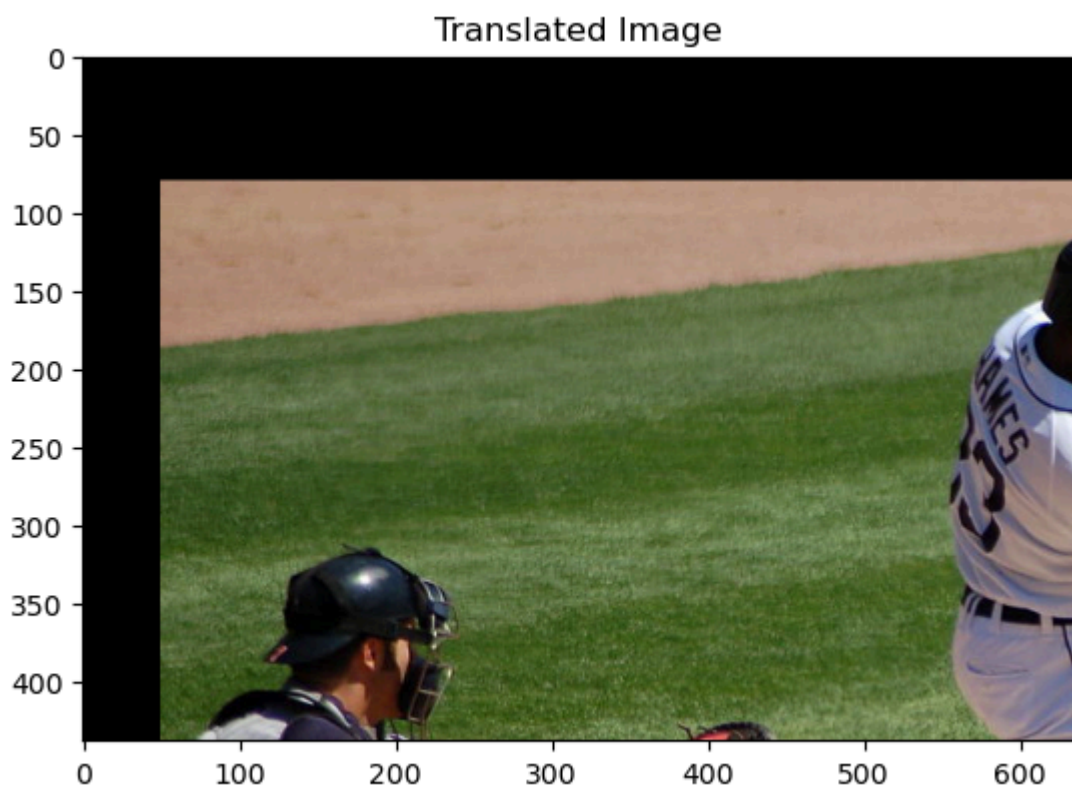
```
Out[2]: (671, 1090, 3)
```

```
In [4]: # Display the image using Matplotlib
plt.imshow(image[:, :, ::-1])
plt.title('Original Image')
plt.show()
```



```
In [7]: #Image Translation
tx, ty = 50, 80 # Translation factors (shift by 50 pixels horizontally and 80 v
M_translation = np.float32([[1, 0, tx], [0, 1, ty]]) # Translation matrix:
translated_image = cv2.warpAffine(image, M_translation, (636, 438))
```

```
In [8]: plt.imshow(translated_image[:, :, ::-1])
plt.title("Translated Image")
plt.show()
```



```
In [9]: #Display the Images via SubPlot
plt.figure(figsize=(10, 5))
# First Image
```

```
plt.subplot(1, 2, 1)
plt.imshow(image[:, :, :-1])
plt.title("Original Image")

# Second Image
plt.subplot(1, 2, 2)
plt.imshow(translated_image[:, :, :-1])
plt.title("Translated Image")

plt.tight_layout()
plt.show()
```

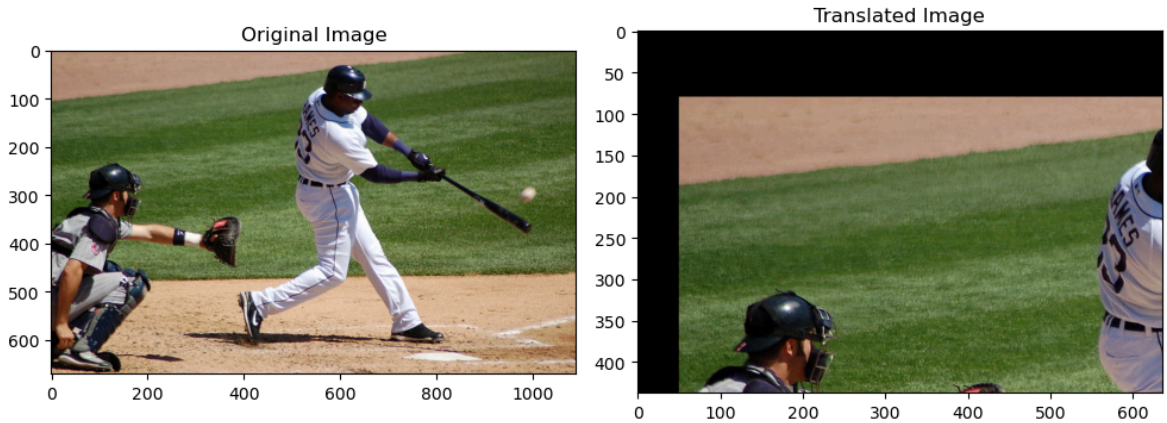


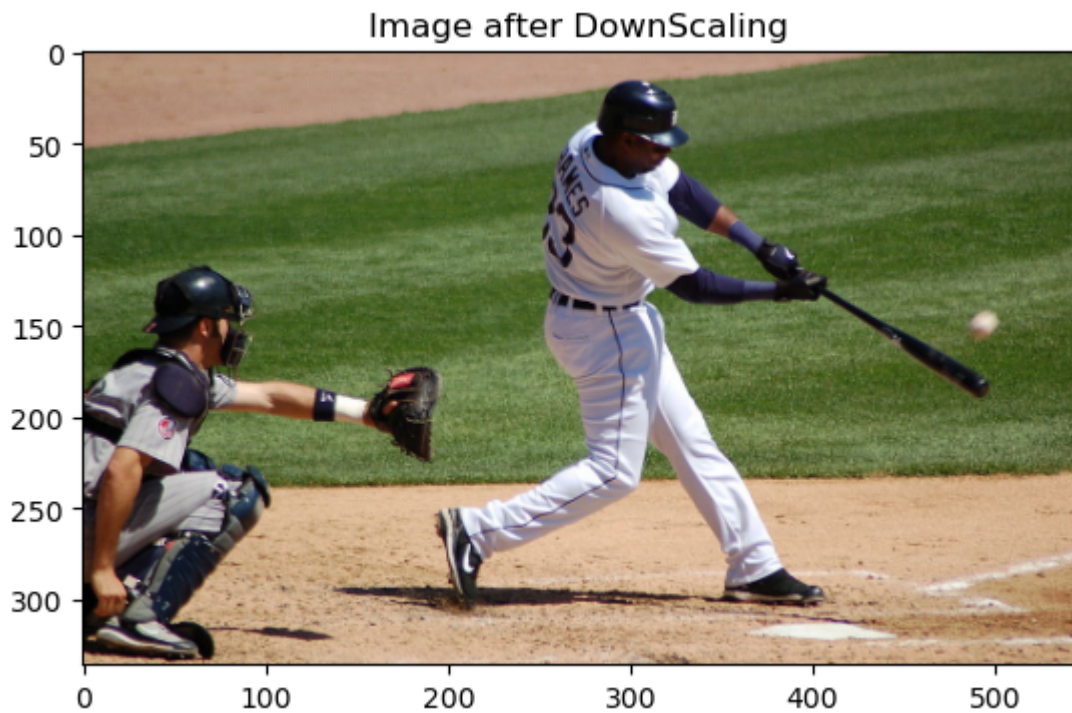
Image Scaling

Write a Python program using OpenCV to perform image scaling on the **given image**.

The program should:

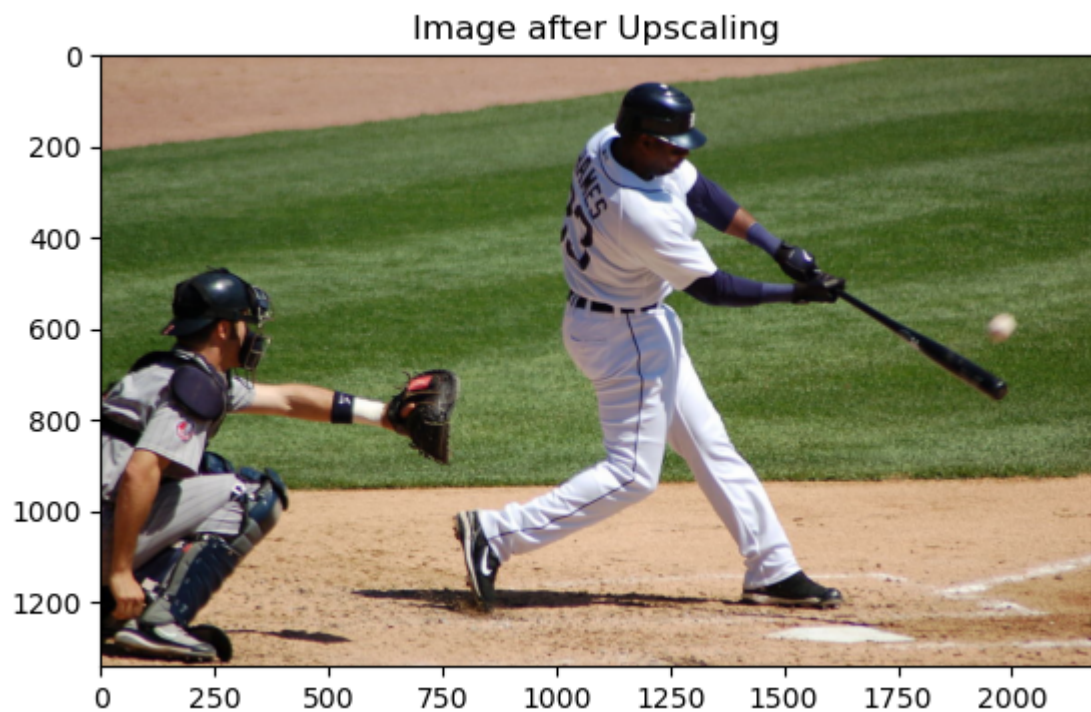
1. Scale the image to:
 - **0.5 times** its original size (downscaling)
 - **2 times** its original size (upscaling)
2. Use `cv2.resize` with appropriate scaling factors.
3. Display the following using a **1 × 3 layout** in Matplotlib:
 - **Original Image**
 - **Downscaled Image (0.5×)**
 - **Upscaled Image (2×)**

```
In [10]: # DownScaling
fx, fy = 0.5, 0.5
Downscaled = cv2.resize(image, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR)
plt.imshow(Downscaled[:, :, :-1])
plt.title("Image after DownScaling")
plt.show()
```



```
In [11]: #Upscaling
fx, fy = 2.0, 2.0
Upscaled = cv2.resize(image, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR)
```

```
In [12]: plt.imshow(Upscaled[:,:,:-1])
plt.title("Image after Upscaling")
plt.show()
```



```
In [13]: #Display the Images
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image[:,:,:-1])
plt.title("Original Image")
```

```
plt.subplot(1, 3, 2)
plt.imshow(Downscaled[:, :, ::-1])
plt.title("0.5x Downscaled")

plt.subplot(1, 3, 3)
plt.imshow(Upscaled[:, :, ::-1])
plt.title("2x Upscaled")

plt.tight_layout()
plt.show()
```

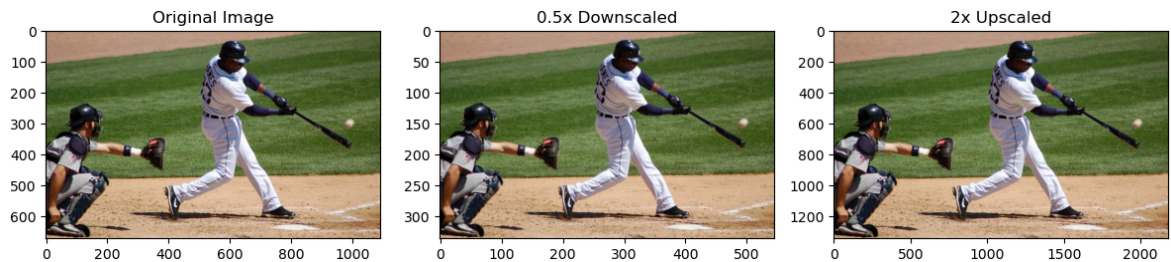


Image Shearing

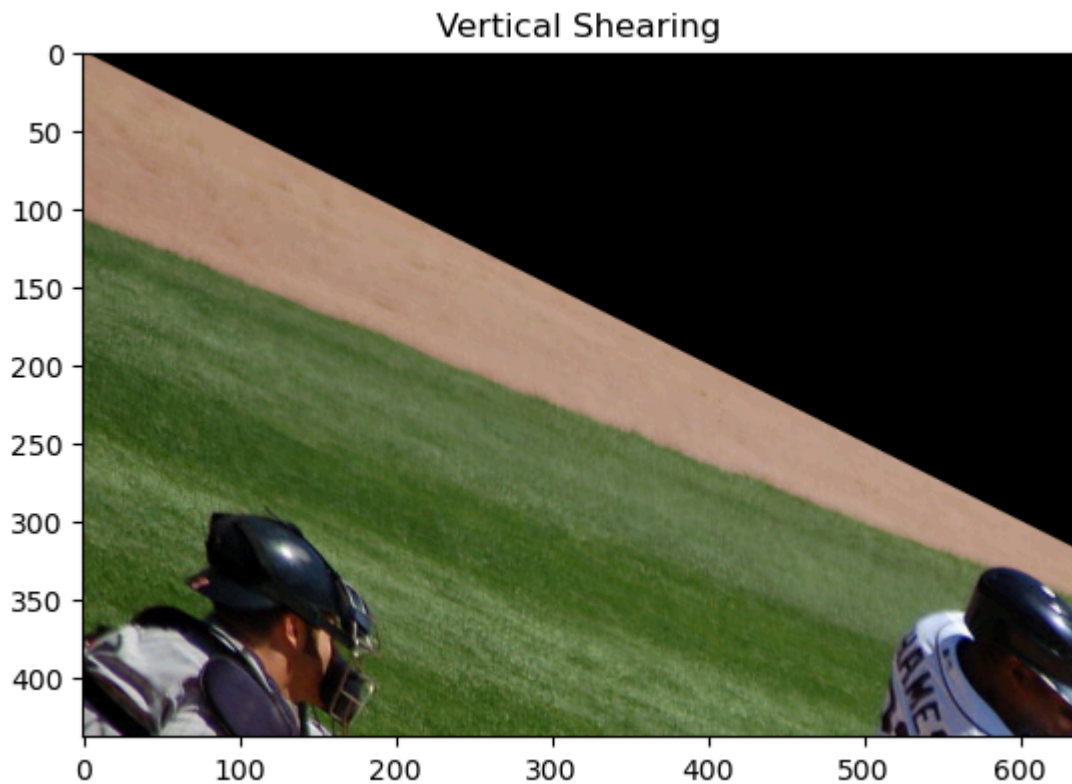
Write a Python program using OpenCV to perform image shearing on the **given image**.

The program should:

1. Create a **shearing transformation matrix** to apply:
 - **Horizontal shearing**
 - **Vertical shearing**
2. Apply the shearing transformations.
3. Display the following using a **1 × 3 layout** in Matplotlib:
 - **Original Image**
 - **Horizontally Sheared Image**
 - **Vertically Sheared Image**

```
In [ ]: # Display Horizontal Shearing
shear_matrix = np.float32([[1, 0.5, 0], [0.5, 1, 0]])
sheared_image = cv2.warpAffine(image, shear_matrix, (636, 438))
plt.imshow(sheared_image[:, :, ::-1])
plt.title("Horizontal Shearing")
plt.show()
```

```
In [14]: # Display Vertical Shearing
shear_matrix = np.float32([[1, 0, 0], [0.5, 1, 0]])
vsheared_image = cv2.warpAffine(image, shear_matrix, (636, 438))
plt.imshow(vsheared_image[:, :, ::-1])
plt.title("Vertical Shearing")
plt.show()
```



```
In [19]: #Display the Images
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image[:, :, ::-1])
plt.title("Original Image")

plt.subplot(1, 3, 2)
plt.imshow(vsheared_image[:, :, ::-1])
plt.title("Horizontal Shearing")

plt.subplot(1, 3, 3)
plt.imshow(vsheared_image[:, :, ::-1])
plt.title("Vertical Shearing")

plt.tight_layout()
plt.show()
```

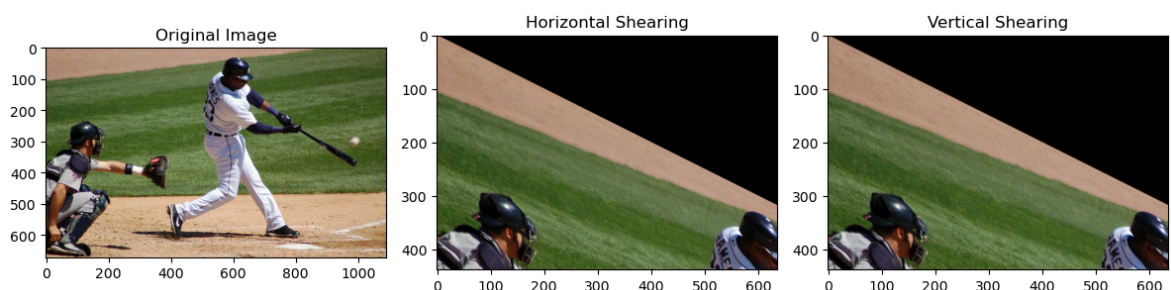


Image Reflection

Write a Python program using OpenCV to perform image reflection on the **given image**.

The program should:

1. Generate the following reflected versions of the image:
 - **Horizontal Reflection** (flip left ↔ right)
 - **Vertical Reflection** (flip top ↔ bottom)
 - **Both-axis Reflection**
2. Use `cv2.flip` with appropriate flip codes.
3. Display the following using a **2 × 2 layout** in Matplotlib:
 - **Original Image**
 - **Horizontally Reflected Image**
 - **Vertically Reflected Image**
 - **Both-axis Reflected Image**

```
In [20]: # Image Reflection
hreflected_image = cv2.flip(image, 1)
vreflected_image = cv2.flip(image, 0)
areflected_image = cv2.flip(image, -1)
```

```
In [21]: plt.imshow(hreflected_image[:,::-1])
plt.title("Horizontal Reflected Image")
plt.axis('off')
```

```
Out[21]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

Horizontal Reflected Image



```
In [22]: plt.imshow(vreflected_image[:,::-1])
plt.title("Vertical Reflected Image")
plt.axis('off')
```

```
Out[22]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

Vertical Reflected Image



```
In [23]: plt.imshow(areflected_image[:, :, ::-1])  
plt.title("Both axis Reflected Image")  
plt.axis('off')
```

```
Out[23]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

Both axis Reflected Image



```
In [24]: plt.figure(figsize=(10, 8))  
  
# Original Image  
plt.subplot(2, 2, 1)  
plt.imshow(image[:, :, ::-1])  
plt.title("Original Image")  
plt.axis("off")  
  
# Horizontal Reflection  
plt.subplot(2, 2, 2)  
plt.imshow(hreflected_image[:, :, ::-1])  
plt.title("Horizontal Reflection")
```

```
plt.axis("off")

# Vertical Reflection
plt.subplot(2, 2, 3)
plt.imshow(vreflected_image[:, :, ::-1])
plt.title("Vertical Reflection")
plt.axis("off")

#Both axis reflected Image
plt.subplot(2, 2, 4)
plt.imshow(areflected_image[:, :, ::-1])
plt.title("Both axis Reflection")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Horizontal Reflection



Vertical Reflection



Both axis Reflection



Image Rotation

Write a Python program using OpenCV to perform **image rotation** on the **given image**.

The program should:

1. Rotate the image by:
 - **45 degrees**
 - **90 degrees**
2. Use `cv2.getRotationMatrix2D` and `cv2.warpAffine`.
3. Display the following using a **1 × 3 layout** in Matplotlib:
 - **Original Image**
 - **45° Rotated Image**

- **90° Rotated Image**

```
In [25]: (height, width) = image.shape[:2]
angle = 45
center = (width // 2, height // 2)
M_rotation = cv2.getRotationMatrix2D(center, angle, 1)
rotated_image = cv2.warpAffine(image, M_rotation, (width, height))
```

```
In [26]: plt.imshow(rotated_image[:, :, ::-1])
plt.title("45 degree Rotated Image")
plt.axis('off')
```

```
Out[26]: (np.float64(-0.5), np.float64(1089.5), np.float64(670.5), np.float64(-0.5))
```

45 degree Rotated Image



```
In [27]: (height_90, width_90) = image.shape[:2]
angle_90 = 90
center_90 = (width_90 // 2, height_90 // 2)
rotation_matrix_90 = cv2.getRotationMatrix2D(center_90, angle_90, 1)
rotated_image_90 = cv2.warpAffine(image, rotation_matrix_90, (width_90, height_90))
```

```
In [28]: # Display the rotated image
plt.imshow(rotated_image_90[:, :, ::-1])
plt.title("90 degree Rotated Image")
plt.axis("off")
plt.show()
```

90 degree Rotated Image



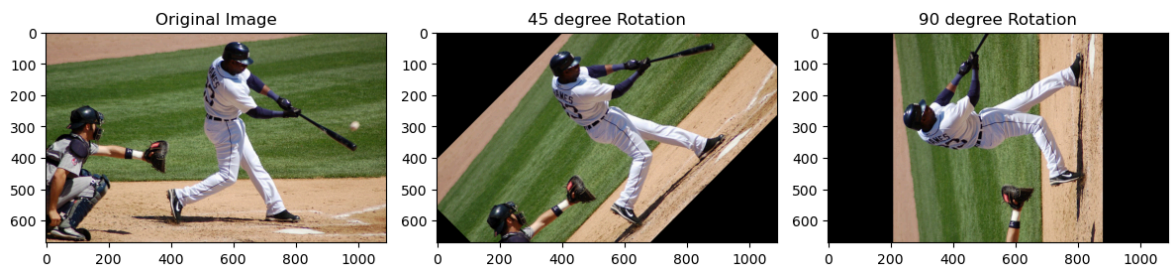
```
In [29]: #Display the Images
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image[:, ::-1])
plt.title("Original Image")

plt.subplot(1, 3, 2)
plt.imshow(rotated_image[:, ::-1])
plt.title("45 degree Rotation")

plt.subplot(1, 3, 3)
plt.imshow(rotated_image_90[:, ::-1])
plt.title("90 degree Rotation")

plt.tight_layout()
plt.show()
```



In []:

In []: